USER PROMPT:

You are a senior Go backend engineer.

Problem: We need to design a URL shortener service in Go.

Use the Least-to-Most Prompting method:
1. Start by listing the subproblems that need to be solved as a to-do list. Use markdown checkboxes: [ ] for pending, [x] for completed.
2. As you solve each subproblem, update the to-do list by marking it as [x] and write the solution right below it.
3. Continue until all items are solved.
4. At the end, combine all the solutions into a final integrated design for the URL shortener.

Constraints:
- Service must be implemented in Go.
- Short URLs must be unique and easy to generate.
- Must support endpoints: shorten a URL, retrieve the original URL.
- Use an in-memory store at first, but mention how it could scale with a database.
- Include minimal validation and error handling.
- Keep explanations concise and structured.

Output format:
- To-do list with checkboxes (updating as you progress)
- Each solved subproblem explained with reasoning and minimal Go code snippets
- Final combined design

LLM RESPONSE:
Okay, let's design this URL shortener service in Go.

To-Do List

[x] 1. Define the core data structure for URL mappings.
[x] 2. Design and implement a unique short code generation mechanism.
[x] 3. Implement an in-memory storage for URL mappings, considering concurrency.
[x] 4. Implement the "shorten URL" core logic.
[x] 5. Implement the "retrieve original URL" core logic.
[x] 6. Design the HTTP API endpoints for shortening and retrieving.
[x] 7. Implement the HTTP server and routing using Go's `net/http` package.
[x] 8. Add minimal validation and error handling for API requests.
[x] 9. Discuss how to scale the storage with a persistent database.
[x] 10. Combine all components into a final integrated design.

Solved Subproblems

[x] 1. Define the core data structure for URL mappings.

Reasoning: We need a way to store the relationship between a short code and its corresponding original (long) URL. A simple struct will suffice, potentially including a timestamp for future features like expiry or analytics.

Go Code Snippet:

```go
package main

import "time"

// URLMapping represents a single URL mapping entry.
type URLMapping struct {
        ShortCode   string    `json:"short_code"`
        OriginalURL string    `json:"original_url"`
        CreatedAt   time.Time `json:"created_at"`
}
```

---

[x] 2. Design and implement a unique short code generation mechanism.

Reasoning: Short codes must be unique and relatively short. A common approach is to use a counter and encode it into a base (e.g., Base62, which uses `0-9a-zA-Z`). This guarantees uniqueness (as long as the counter is unique) and produces concise codes. We'll use a simple counter and Base62 encoding.

Go Code Snippet:

```go
package main

import (
	"math/rand"
	"strings"
	"sync"
	"time"
)

// Base62 characters
const base62Chars = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"

var (
	// For unique ID generation (could be a distributed counter in a real system)
	counter     int64
	counterLock sync.Mutex
	// For random part in case a counter isn't strictly sequential or for added entropy
	rnd = rand.New(rand.NewSource(time.Now().UnixNano()))
)

// generateShortCode generates a unique short code using a combination of counter and base62 encoding.
func generateShortCode() string {
	counterLock.Lock()
	counter++
	currentCounter := counter
	counterLock.Unlock()

	// Convert counter to Base62
	var sb strings.Builder
	if currentCounter == 0 {
		return string(base62Chars[0]) // Handle 0 explicitly
	}
	for currentCounter > 0 {
		sb.WriteByte(base62Chars[currentCounter%62])
		currentCounter /= 62
	}
	return reverseString(sb.String())
}

// reverseString reverses a string (helper for base62 conversion)
func reverseString(s string) string {
	runes := []rune(s)
	for i, j := 0, len(runes)-1; i < j; i, j = i+1, j-1 {
		runes[i], runes[j] = runes[j], runes[i]
	}
	return string(runes)
}
```

---

[x] 3. Implement an in-memory storage for URL mappings, considering concurrency.

Reasoning: An `in-memory map` is perfect for a fast, simple store. Since multiple goroutines (HTTP requests) will access and modify this map, a `sync.RWMutex` is essential to prevent race conditions. Read-write mutex allows multiple readers concurrently but only one writer.

Go Code Snippet:

```go
package main

import (
	"errors"
	"sync"
	"time"
)

// URLStore manages the in-memory storage for URL mappings.
type URLStore struct {
	mu      sync.RWMutex
	mappings map[string]URLMapping // shortCode -> URLMapping
	// Could also have originalURL -> shortCode for fast lookups if we need to prevent duplicate shortens for the same long URL
	// originalToShort map[string]string
}

// NewURLStore creates and initializes a new URLStore.
```

```
func NewURLStore() *URLStore {
	return &URLStore{
		mappings: make(map[string]URLMapping),
	}
}

// Save stores a new URL mapping.
func (s *URLStore) Save(mapping URLMapping) error {
	s.mu.Lock()
	defer s.mu.Unlock()

	if _, exists := s.mappings[mapping.ShortCode]; exists {
		return errors.New("short code already exists") // Should not happen with unique generator
	}
	s.mappings[mapping.ShortCode] = mapping
	return nil
}

// Get retrieves a URL mapping by its short code.
func (s *URLStore) Get(shortCode string) (URLMapping, error) {
	s.mu.RLock()
	defer s.mu.RUnlock()

	mapping, exists := s.mappings[shortCode]
	if !exists {
		return URLMapping{}, errors.New("short code not found")
	}
	return mapping, nil
}
```

---

[x] 4. Implement the "shorten URL" core logic.

Reasoning: This function will orchestrate the short code generation and storage. It takes an original URL, generates a unique short code, creates a `URLMapping` object, and saves it to the store. It should also perform basic validation on the input URL.

Go Code Snippet:

```go
package main

import (
	"errors"
	"net/url"
	"time"
)

// ShortenerService encapsulates the logic for shortening URLs.
type ShortenerService struct {
	store *URLStore
	baseURL string // The base URL for the shortener service itself, e.g., "http://localhost:8080/"
}

// NewShortenerService creates a new ShortenerService.
func NewShortenerService(store *URLStore, baseURL string) *ShortenerService {
	return &ShortenerService{
		store: store,
		baseURL: baseURL,
	}
}

// ShortenURL takes an original URL and returns a short URL.
func (s *ShortenerService) ShortenURL(originalURL string) (string, error) {
	// Basic validation: Check if it's a valid URL format
	parsedURL, err := url.ParseRequestURI(originalURL)
	if err != nil || parsedURL.Scheme == "" || parsedURL.Host == "" {
		return "", errors.New("invalid original URL format")
	}

	shortCode := generateShortCode()
	mapping := URLMapping{
		ShortCode:   shortCode,
		OriginalURL: originalURL,
		CreatedAt:   time.Now(),
	}

	if err := s.store.Save(mapping); err != nil {
```

```
                return "", err // Propagate storage error
        }

        return s.baseURL + shortCode, nil
}
```

---

[x] 5. Implement the "retrieve original URL" core logic.

Reasoning: This function will take a short code, look it up in the store, and return the original URL. If the short code is not found, it should return an error.

Go Code Snippet:

```go
package main

import (
        "errors"
)

// GetOriginalURL retrieves the original URL for a given short code.
func (s *ShortenerService) GetOriginalURL(shortCode string) (string, error) {
        if shortCode == "" {
                return "", errors.New("short code cannot be empty")
        }

        mapping, err := s.store.Get(shortCode)
        if err != nil {
                return "", err // Propagate "not found" or other storage errors
        }
        return mapping.OriginalURL, nil
}
```

---

[x] 6. Design the HTTP API endpoints for shortening and retrieving.

Reasoning:
*   **Shorten:** A `POST` request to `/shorten` is appropriate, with the original URL in the request body (JSON). This is a creation operation.
*   **Retrieve/Redirect:** A `GET` request to `/{short_code}` is standard. The server should respond with an HTTP 302 Found redirect to the original URL.

Endpoints:

*   `POST /shorten`
    *   **Request Body (JSON):** `{"long_url": "https://example.com/very/long/url"}`
    *   **Response Body (JSON):** `{"short_url": "http://localhost:8080/abcde"}` or error.
*   `GET /{short_code}`
    *   **Example:** `GET /abcde`
    *   **Response:** HTTP 302 Redirect to `https://example.com/very/long/url` or HTTP 404 if not found.

---

[x] 7. Implement the HTTP server and routing using Go's `net/http` package.

Reasoning: Go's standard `net/http` package provides all the necessary tools. We'll define handler functions for each endpoint and use `http.HandleFunc` to register them with the default server mux.

Go Code Snippet:

```go
package main

import (
        "encoding/json"
        "fmt"
        "log"
        "net/http"
        "strings"
)

// Request and Response structs for JSON marshalling/unmarshalling
type ShortenRequest struct {
        LongURL string `json:"long_url"`
}
```

```go
type ShortenResponse struct {
	ShortURL string `json:"short_url"`
}

type ErrorResponse struct {
	Error string `json:"error"`
}

// ShortenHandler handles POST requests to /shorten
func (s *ShortenerService) ShortenHandler(w http.ResponseWriter, r *http.Request) {
	if r.Method != http.MethodPost {
		http.Error(w, "Method Not Allowed", http.StatusMethodNotAllowed)
		return
	}

	var req ShortenRequest
	if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
		writeJSONError(w, "Invalid request body", http.StatusBadRequest)
		return
	}

	shortURL, err := s.ShortenURL(req.LongURL)
	if err != nil {
		log.Printf("Error shortening URL '%s': %v", req.LongURL, err)
		if strings.Contains(err.Error(), "invalid original URL") {
			writeJSONError(w, err.Error(), http.StatusBadRequest)
		} else {
			writeJSONError(w, "Failed to shorten URL", http.StatusInternalServerError)
		}
		return
	}

	resp := ShortenResponse{ShortURL: shortURL}
	writeJSONResponse(w, resp, http.StatusCreated)
}

// RedirectHandler handles GET requests to /{short_code}
func (s *ShortenerService) RedirectHandler(w http.ResponseWriter, r *http.Request) {
	if r.Method != http.MethodGet {
		http.Error(w, "Method Not Allowed", http.StatusMethodNotAllowed)
		return
	}

	shortCode := strings.TrimPrefix(r.URL.Path, "/")
	if shortCode == "" {
		http.NotFound(w, r)
		return
	}

	originalURL, err := s.GetOriginalURL(shortCode)
	if err != nil {
		if strings.Contains(err.Error(), "not found") {
			http.NotFound(w, r)
		} else {
			log.Printf("Error retrieving original URL for '%s': %v", shortCode, err)
			http.Error(w, "Internal Server Error", http.StatusInternalServerError)
		}
		return
	}

	http.Redirect(w, r, originalURL, http.StatusFound) // 302 Found
}

// Helper to write JSON responses
func writeJSONResponse(w http.ResponseWriter, data interface{}, statusCode int) {
	w.Header().Set("Content-Type", "application/json")
	w.WriteHeader(statusCode)
	if err := json.NewEncoder(w).Encode(data); err != nil {
		log.Printf("Error writing JSON response: %v", err)
	}
}

// Helper to write JSON error responses
func writeJSONError(w http.ResponseWriter, message string, statusCode int) {
	writeJSONResponse(w, ErrorResponse{Error: message}, statusCode)
}

// Setup and run the server (will be in main function)
func main() {
```

```
        // ... (setup store and service)
        // http.HandleFunc("/shorten", service.ShortenHandler)
        // http.HandleFunc("/", service.RedirectHandler) // This will catch all other paths
        // log.Fatal(http.ListenAndServe(":8080", nil))
}
```

---

[x] 8. Add minimal validation and error handling for API requests.

Reasoning: As seen in the handlers above, we need to:
*   Validate HTTP methods.
*   Validate request body (JSON decoding errors, empty `long_url`).
*   Validate `long_url` format (using `net/url`).
*   Handle cases where short codes are not found.
*   Return appropriate HTTP status codes and informative JSON error messages.
*   Log internal errors for debugging.

Go Code Snippet: (Already integrated into `ShortenHandler`, `RedirectHandler`, and helper functions in Subproblem 7)

---

[x] 9. Discuss how to scale the storage with a persistent database.

Reasoning: The current in-memory store is great for a prototype but loses all data on restart and doesn't scale horizontally. A persistent database is essential for production.

Discussion:

1. **Replace `URLStore`:** The `URLStore` interface (or struct methods) would remain largely the same (`Save`, `Get`), but the underlying implementation would change to interact with a database.
2. **Database Choices:**
    *   **SQL (e.g., PostgreSQL, MySQL):**
        *   **Schema:** A simple table like `CREATE TABLE url_mappings (short_code VARCHAR(20) PRIMARY KEY, original_url TEXT NOT NULL, created_at TIMESTAMP NOT NULL);`
        *   **Pros:** Strong consistency, ACID properties, mature ecosystem, good for querying if analytics are added.
        *   **Cons:** Can become a bottleneck for extremely high write throughput if not sharded/scaled properly.
        *   **Go Integration:** Use `database/sql` package with a specific driver (e.g., `github.com/lib/pq`).
    *   **NoSQL (e.g., Redis, Cassandra, DynamoDB):**
        *   **Redis:** Excellent for caching and key-value lookups. Could be used as a primary store for short-lived URLs or as a cache layer in front of a slower persistent store.
            *   **Pros:** Extremely fast reads/writes.
            *   **Cons:** Data can be lost if not configured for persistence (RDB/AOF), single-threaded nature can be a bottleneck for complex operations.
        *   **Cassandra/DynamoDB (Wide-Column/Document):**
            *   **Pros:** Highly scalable, distributed, high availability, good for very large datasets and high write throughput.
            *   **Cons:** Eventual consistency (depending on configuration), steeper learning curve, less flexible querying than SQL.
        *   **Go Integration:** Use specific client libraries (e.g., `github.com/go-redis/redis/v8`, `github.com/aws/aws-sdk-go`).
3. **Short Code Generation with DB:**
    *   **Counter in DB:** The `counter` could be stored in the database and incremented atomically. This ensures global uniqueness across multiple service instances.
    *   **Random String:** Alternatively, generate a random string (e.g., 6-8 alphanumeric characters) and attempt to save it. If a collision occurs (due to random chance), retry with a new random string. This is common with services like TinyURL.
4. **Concurrency:** Database drivers handle connection pooling and concurrency. `sync.RWMutex` would no longer be needed around the database client itself, as the database handles its own concurrency control.

For this service, a simple SQL database (like PostgreSQL) is a robust and common choice.

---

[x] 10. Combine all components into a final integrated design.

Reasoning: Assemble all the pieces into a runnable Go application (`main.go`).

Final Integrated Design (main.go):

```go
package main

import (
        "encoding/json"
        "errors"
        "fmt"
```

```
	"log"
	"math/rand"
	"net/http"
	"net/url"
	"strings"
	"sync"
	"time"
)

// --- 1. Core Data Structure ---

// URLMapping represents a single URL mapping entry.
type URLMapping struct {
	ShortCode   string    `json:"short_code"`
	OriginalURL string    `json:"original_url"`
	CreatedAt   time.Time `json:"created_at"`
}

// --- 2. Short Code Generation ---

// Base62 characters
const base62Chars = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"

var (
	// Global counter for unique ID generation. In a distributed system, this would be a distributed counter.
	counter     int64
	counterLock sync.Mutex
)

// generateShortCode generates a unique short code using a counter and base62 encoding.
func generateShortCode() string {
	counterLock.Lock()
	counter++
	currentCounter := counter
	counterLock.Unlock()

	// Convert counter to Base62
	var sb strings.Builder
	if currentCounter == 0 {
		return string(base62Chars[0]) // Handle 0 explicitly
	}
	for currentCounter > 0 {
		sb.WriteByte(base62Chars[currentCounter%62])
		currentCounter /= 62
	}
	return reverseString(sb.String())
}

// reverseString reverses a string (helper for base62 conversion)
func reverseString(s string) string {
	runes := []rune(s)
	for i, j := 0, len(runes)-1; i < j; i, j = i+1, j-1 {
		runes[i], runes[j] = runes[j], runes[i]
	}
	return string(runes)
}

// --- 3. In-Memory Storage ---

// URLStore manages the in-memory storage for URL mappings.
type URLStore struct {
	mu       sync.RWMutex
	mappings map[string]URLMapping // shortCode -> URLMapping
}

// NewURLStore creates and initializes a new URLStore.
func NewURLStore() *URLStore {
	return &URLStore{
		mappings: make(map[string]URLMapping),
	}
}

// Save stores a new URL mapping.
func (s *URLStore) Save(mapping URLMapping) error {
	s.mu.Lock()
	defer s.mu.Unlock()

	if _, exists := s.mappings[mapping.ShortCode]; exists {
		// This should ideally not happen with a sequential counter unless manually added.
		return errors.New("short code already exists")
```

```
        }
        s.mappings[mapping.ShortCode] = mapping
        return nil
}

// Get retrieves a URL mapping by its short code.
func (s *URLStore) Get(shortCode string) (URLMapping, error) {
        s.mu.RLock()
        defer s.mu.RUnlock()

        mapping, exists := s.mappings[shortCode]
        if !exists {
                return URLMapping{}, errors.New("short code not found")
        }
        return mapping, nil
}

// --- 4 & 5. Shortener Service Logic ---

// ShortenerService encapsulates the logic for shortening and retrieving URLs.
type ShortenerService struct {
        store   *URLStore
        baseURL string // The base URL for the shortener service itself, e.g., "http://localhost:8080/"
}

// NewShortenerService creates a new ShortenerService.
func NewShortenerService(store *URLStore, baseURL string) *ShortenerService {
        return &ShortenerService{
                store:   store,
                baseURL: baseURL,
        }
}

// ShortenURL takes an original URL and returns a short URL.
func (s *ShortenerService) ShortenURL(originalURL string) (string, error) {
        // Basic validation: Check if it's a valid URL format
        parsedURL, err := url.ParseRequestURI(originalURL)
        if err != nil || parsedURL.Scheme == "" || parsedURL.Host == "" {
                return "", errors.New("invalid original URL format")
        }

        shortCode := generateShortCode()
        mapping := URLMapping{
                ShortCode:   shortCode,
                OriginalURL: originalURL,
                CreatedAt:   time.Now(),
        }

        if err := s.store.Save(mapping); err != nil {
                return "", fmt.Errorf("failed to save URL mapping: %w", err)
        }

        return s.baseURL + shortCode, nil
}

// GetOriginalURL retrieves the original URL for a given short code.
func (s *ShortenerService) GetOriginalURL(shortCode string) (string, error) {
        if shortCode == "" {
                return "", errors.New("short code cannot be empty")
        }

        mapping, err := s.store.Get(shortCode)
        if err != nil {
                return "", fmt.Errorf("failed to retrieve URL mapping: %w", err)
        }
        return mapping.OriginalURL, nil
}

// --- 6, 7 & 8. HTTP API Endpoints, Server, Routing, Validation & Error Handling ---

// Request and Response structs for JSON marshalling/unmarshalling
type ShortenRequest struct {
        LongURL string `json:"long_url"`
}

type ShortenResponse struct {
        ShortURL string `json:"short_url"`
}

type ErrorResponse struct {
```

```
	Error string `json:"error"`
}

// ShortenHandler handles POST requests to /shorten
func (s *ShortenerService) ShortenHandler(w http.ResponseWriter, r *http.Request) {
	if r.Method != http.MethodPost {
		http.Error(w, "Method Not Allowed", http.StatusMethodNotAllowed)
		return
	}

	var req ShortenRequest
	if err := json.NewDecoder(r.Body).Decode(&req); err != nil {
		writeJSONError(w, "Invalid request body", http.StatusBadRequest)
		return
	}

	shortURL, err := s.ShortenURL(req.LongURL)
	if err != nil {
		log.Printf("Error shortening URL '%s': %v", req.LongURL, err)
		if strings.Contains(err.Error(), "invalid original URL") {
			writeJSONError(w, err.Error(), http.StatusBadRequest)
		} else {
			writeJSONError(w, "Failed to shorten URL", http.StatusInternalServerError)
		}
		return
	}

	resp := ShortenResponse{ShortURL: shortURL}
	writeJSONResponse(w, resp, http.StatusCreated)
}

// RedirectHandler handles GET requests to /{short_code}
func (s *ShortenerService) RedirectHandler(w http.ResponseWriter, r *http.Request) {
	if r.Method != http.MethodGet {
		http.Error(w, "Method Not Allowed", http.StatusMethodNotAllowed)
		return
	}

	// Extract short code from path. e.g., for /abcde, shortCode will be "abcde"
	// We need to trim the leading slash carefully for the root path.
	shortCode := strings.TrimPrefix(r.URL.Path, "/")
	if shortCode == "" {
		http.NotFound(w, r) // If path is just "/" or empty, it's not a short code.
		return
	}

	originalURL, err := s.GetOriginalURL(shortCode)
	if err != nil {
		if errors.Is(err, errors.New("failed to retrieve URL mapping: short code not found")) { // Check
specific error
			http.NotFound(w, r)
		} else {
			log.Printf("Error retrieving original URL for '%s': %v", shortCode, err)
			http.Error(w, "Internal Server Error", http.StatusInternalServerError)
		}
		return
	}

	http.Redirect(w, r, originalURL, http.StatusFound) // 302 Found
}

// Helper to write JSON responses
func writeJSONResponse(w http.ResponseWriter, data interface{}, statusCode int) {
	w.Header().Set("Content-Type", "application/json")
	w.WriteHeader(statusCode)
	if err := json.NewEncoder(w).Encode(data); err != nil {
		log.Printf("Error writing JSON response: %v", err)
	}
}

// Helper to write JSON error responses
func writeJSONError(w http.ResponseWriter, message string, statusCode int) {
	writeJSONResponse(w, ErrorResponse{Error: message}, statusCode)
}

func main() {
	// Initialize random source for potential future random short code generation
	rand.New(rand.NewSource(time.Now().UnixNano()))

	// --- Service Initialization ---
```

```
        store := NewURLStore()
        service := NewShortenerService(store, "http://localhost:8080/") // Base URL of the shortener service

        // --- HTTP Server Setup ---
        http.HandleFunc("/shorten", service.ShortenHandler)
        http.HandleFunc("/", service.RedirectHandler) // This should be the last handler as it catches all paths

        port := ":8080"
        log.Printf("URL Shortener service starting on port %s", port)
        log.Printf("To shorten a URL: POST %s/shorten with JSON body {\"long_url\": \"your_long_url_here\"}",
service.baseURL)
        log.Printf("To retrieve original URL: GET %s/{short_code}", service.baseURL)

        log.Fatal(http.ListenAndServe(port, nil)) // Start the HTTP server
}
```

```
Model: gemini-2.5-flash
Token usage (estimated):
  Input tokens:  261
  Output tokens: 5370
  Total tokens:  5631
-----------------------------------------------------
```